

Chapter 6: Structures

Grouping heterogeneous data under one name -- and pointing structs at each other.

How to use this document

Read the graph first. Each node is a concept; each edge is a dependency. Walk the edges in order. After each edge, do the green problem immediately. Checkpoints (red) tell you whether you can proceed. The target problem at the end ties every edge together. Answer key is on the last page.

Big picture

A struct bundles variables of different types into one named unit. You access members with `.` (dot) or `->` (arrow, for pointers). Once a struct can contain a pointer to its own type, you get linked lists and trees -- the backbone of symbol tables, compilers, and most non-trivial C programs.

Target problem

Build a symbol table using a self-referential struct (binary tree or linked list) with `insert`, `lookup`, and `print` operations.

By the end of this document you will write this from scratch, understanding every line.

The Graph

Node inventory

Node	What it is	Why it matters
struct	A named bundle of typed members	Groups related data into one unit
<code>.</code> (dot)	Member access on a struct value	The primary way to read/write fields
<code>-></code> (arrow)	Member access through a pointer	Equivalent to <code>(*p).member</code> -- less error-prone
typedef	Creates an alias for an existing type	Hides complex declarations, aids portability
union	Overlays multiple types in the same memory	Saves space when only one variant is active at a time

bit-field	Integer members with explicit bit widths	Compact flag storage inside a struct
self-referential struct	A struct with a pointer to its own type	Enables linked lists, trees, graphs
malloc	Allocates heap memory, returns void*	Creates nodes at runtime for dynamic structures

Edge table

From	To	Relationship
struct	. / ->	Members are accessed via dot or arrow
struct	typedef	typedef creates a short alias for a struct type
struct	union	union uses struct-like syntax but overlays members
struct	bit-field	bit-fields are struct members with : width
pointer-to-struct	->	Arrow dereferences and accesses in one step
pointer-to-struct	self-referential struct	A struct holds a pointer to its own type -- recursion in data
self-referential struct	malloc	Each new node is heap-allocated at runtime

Zoom in -- one edge at a time

Edge 1: struct contains members, accessed with dot

A struct declaration is a blueprint. A struct variable is an instance that occupies memory. You reach each field with the dot operator.

```
struct sensor {
    char name[32];
    double reading;
    int active;
};

struct sensor s1 = {"temp_01", 23.5, 1};
printf("%s: %.1f\n", s1.name, s1.reading);
```

Do now: Declare a struct `vec3` with three doubles (`x`, `y`, `z`). Write a function `double mag(struct vec3 v)` that returns the magnitude $\sqrt{x^2 + y^2 + z^2}$. Call it from main and print the result.

Edge 2: pointer-to-struct enables arrow access

Passing large structs by value copies every byte. Passing a pointer is cheap. The arrow operator `p->member` is shorthand for `(*p).member`.

```
void print_sensor(struct sensor *sp) {
    printf("%s: %.1f (active=%d)\n",
        sp->name, sp->reading, sp->active);
}

struct sensor s1 = {"temp_01", 23.5, 1};
print_sensor(&s1);
```

Do now: Write `void scale(struct vec3 *v, double k)` that multiplies each component by `k` in place. Verify with a print.

Edge 3: typedef creates alias for struct types

Writing `struct sensor` everywhere is verbose. `typedef` lets you create a short name. It does not create a new type -- just a synonym.

```
typedef struct sensor Sensor;
/* Now you can write: */
Sensor s2 = {"pressure_03", 101.3, 1};

/* Or combine declaration and typedef: */
typedef struct {
    double real;
    double imag;
} Complex;
```

Do now: Use typedef to create Vec3. Rewrite your mag function signature to use it.

Edge 4: union overlays members in same memory

A union looks like a struct but all members share the same starting address. Its size equals its largest member. You must track which member is currently valid.

```
typedef struct {
    int kind; /* 0=int, 1=float, 2=string */
    union {
        int ival;
        float fval;
        char *sval;
    } val;
} Token;

Token t;
t.kind = 0;
t.val.ival = 42;
```

Do now: Print sizeof(Token). Then change kind to 2, set val.sval to a string literal, and print it. Confirm the size does not change.

Edge 5: bit-fields pack flags into a struct

When you only need a few bits per flag, bit-fields let you declare struct members with explicit widths. They behave like small unsigned integers.

```
struct permissions {
    unsigned int read : 1;
    unsigned int write : 1;
    unsigned int exec : 1;
};

struct permissions p = {1, 1, 0};
if (p.read && !p.exec)
    printf("read-only, no execute\n");
```

Do now: Add a 3-bit priority field (range 0-7) to the permissions struct. Set it to 5 and print it.

CHECKPOINT A

Before continuing, verify you can answer these without looking back:

1. What is the difference between . and -> ?
2. Does typedef create a new type or just an alias?
3. How does a union differ from a struct in memory layout?
4. Can you take the address of a bit-field member?

Edge 6: self-referential struct + malloc = dynamic data structures

A struct cannot contain an instance of itself (infinite size). But it can contain a **pointer** to its own type (fixed size: one address). Combined with malloc, this gives you linked lists, binary trees, and graphs -- all built at runtime.

```
/* Linked-list node */
typedef struct Node {
    char *key;
    int value;
    struct Node *next; /* pointer to own type */
} Node;

Node *new_node(char *k, int v) {
    Node *n = (Node *)malloc(sizeof(Node));
    n->key = strdup(k);
    n->value = v;
    n->next = NULL;
    return n;
}
```

For a binary search tree, each node has two child pointers instead of one next pointer:

```
typedef struct TNode {
    char *word;
    int count;
    struct TNode *left;
    struct TNode *right;
} TNode;
```

Do now: Write a function `Node *prepend(Node *head, char *k, int v)` that creates a new node and makes it the new head of the list. Build a 3-element list and walk it with a for loop, printing each key/value.

Edge 7: hash + linked list = symbol table

A hash table maps a string to an array index. Each slot holds a linked list of entries that share that hash. This gives $O(1)$ average lookup.

```
#define HASHSIZE 101
static Node *hashtab[HASHSIZE];

unsigned hash(char *s) {
    unsigned h = 0;
    for ( ; *s; s++)
        h = *s + 31 * h;
    return h % HASHSIZE;
}

Node *lookup(char *key) {
    Node *np;
    for (np = hashtab[hash(key)]; np; np = np->next)
        if (strcmp(key, np->key) == 0)
            return np;
}
```

```

return NULL;
}

Node *install(char *key, int val) {
Node *np = lookup(key);
if (np == NULL) {
np = new_node(key, val);
unsigned h = hash(key);
np->next = hashtab[h];
hashtab[h] = np;
} else {
np->value = val; /* update */
}
return np;
}

```

Do now: Write a void `dump_table(void)` function that iterates all `HASHSIZE` slots, walks each chain, and prints every key-value pair. Test with 4 installs and 1 overwrite.

Solve the target -- step by step

We will build a symbol table using a binary search tree. Three operations: insert, lookup, print. Each step adds one function.

Step 1: Define the node

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct SymNode {
    char *name;
    char *defn;
    struct SymNode *left;
    struct SymNode *right;
} SymNode;
```

Step 2: Insert (recursive)

```
SymNode *sym_insert(SymNode *root, char *name, char *defn) {
    if (root == NULL) {
        root = (SymNode *)malloc(sizeof(SymNode));
        root->name = strdup(name);
        root->defn = strdup(defn);
        root->left = root->right = NULL;
    } else {
        int cmp = strcmp(name, root->name);
        if (cmp < 0)
            root->left = sym_insert(root->left, name, defn);
        else if (cmp > 0)
            root->right = sym_insert(root->right, name, defn);
        else {
            free(root->defn);
            root->defn = strdup(defn); /* update */
        }
    }
    return root;
}
```

Step 3: Lookup (recursive)

```
SymNode *sym_lookup(SymNode *root, char *name) {
    if (root == NULL) return NULL;
    int cmp = strcmp(name, root->name);
    if (cmp < 0) return sym_lookup(root->left, name);
    if (cmp > 0) return sym_lookup(root->right, name);
    return root; /* found */
}
```

Step 4: Print in order

```

void sym_print(SymNode *root) {
    if (root == NULL) return;
    sym_print(root->left);
    printf("%-20s %s\n", root->name, root->defn);
    sym_print(root->right);
}

```

Step 5: Driver

```

int main(void) {
    SymNode *table = NULL;
    table = sym_insert(table, "PI", "3.14159");
    table = sym_insert(table, "E", "2.71828");
    table = sym_insert(table, "GAMMA", "0.57721");
    table = sym_insert(table, "PI", "3.14159265"); /* update */

    sym_print(table);

    SymNode *found = sym_lookup(table, "E");
    if (found)
        printf("\nLookup E: %s\n", found->defn);
    return 0;
}

```

CHECKPOINT B

1. Why does `sym_insert` return a `SymNode*` instead of `void`?
2. What happens if you insert keys in sorted order? What is the tree shape?
3. Why do we call `strdup` instead of just assigning the pointer?

Extensions

Extension 1: Hash table variant

Replace the BST with a hash table using an array of linked-list chains (as shown in Edge 7 above). The interface stays the same: `insert`, `lookup`, `print`. Average-case lookup drops from $O(\log n)$ to $O(1)$.

Extension 2: Delete operation

Add `SymNode *sym_delete(SymNode *root, char *name)`. Three cases: leaf node (free and return `NULL`), one child (bypass), two children (replace with in-order successor, then delete the successor).

Extension 3: Tagged union values

Change `defn` from `char*` to a tagged union that can hold `int`, `double`, or `char*`. Add a `kind` field. Modify `print` to dispatch on `kind`.

CHECKPOINT C

1. What advantage does a hash table have over a BST for this use case?
2. In BST deletion with two children, why use the in-order successor?
3. If your tagged union holds a `char*`, who owns that memory?

Final exam -- 5 problems

Problem 1. Given this struct:

```
struct pair { int a; int b; };
```

Write a function `struct pair swap(struct pair p)` that returns `p` with `a` and `b` exchanged. Then write a version that takes a pointer and swaps in place.

Problem 2. A linked list of integers. Write: (a) append to tail, (b) reverse the list in place, (c) free all nodes. Draw the pointer diagram for a 3-element list before and after reversal.

Problem 3. Implement a simple phone book using a hash table with separate chaining. Support: `add(name, number)`, `find(name)`, `delete(name)`, and `list_all()`. Use `HASHSIZE=53`.

Problem 4. Define a tagged union type `Value` that can hold an `int`, a `double`, or a `char*`. Write `void print_value(Value v)` that dispatches on the tag. Then build an array of 5 `Values` of mixed types and print them.

Problem 5. Build a word-frequency counter: read `stdin` word by word, insert each word into a BST (incrementing count if it exists), then print all words in alphabetical order with their counts. Test with a paragraph of text via input redirection.

CHECKPOINT D

If you completed all 5 problems, you have working fluency with: struct definition and access, pointer-to-struct with arrow, typedef, union, self-referential structures, malloc/free, linked lists, BSTs, and hash tables. These are the building blocks for every non-trivial C program.

Answer key

Problem 1

```
/* By value */
struct pair swap(struct pair p) {
    int tmp = p.a; p.a = p.b; p.b = tmp;
    return p;
}
/* By pointer */
void swap_ptr(struct pair *p) {
    int tmp = p->a; p->a = p->b; p->b = tmp;
}
```

Problem 2 (key parts)

```
typedef struct INode { int val; struct INode *next; } INode;

INode *append(INode *head, int v) {
    INode *n = malloc(sizeof(INode));
    n->val = v; n->next = NULL;
    if (!head) return n;
    INode *p = head;
    while (p->next) p = p->next;
    p->next = n;
    return head;
}

INode *reverse(INode *head) {
    INode *prev = NULL, *cur = head, *nxt;
    while (cur) {
        nxt = cur->next;
        cur->next = prev;
        prev = cur;
        cur = nxt;
    }
    return prev;
}

void free_list(INode *head) {
    while (head) {
        INode *tmp = head; head = head->next; free(tmp);
    }
}
```

Problem 3 (key parts)

```
#define HASHSIZE 53
typedef struct Entry {
    char *name; char *number;
    struct Entry *next;
} Entry;
static Entry *table[HASHSIZE];
```

```

/* add: insert or update */
void add(char *name, char *number) {
    unsigned h = hash(name);
    Entry *e;
    for (e = table[h]; e; e = e->next)
        if (strcmp(name, e->name) == 0) {
            free(e->number);
            e->number = strdup(number); return;
        }
    e = malloc(sizeof(Entry));
    e->name = strdup(name);
    e->number = strdup(number);
    e->next = table[h]; table[h] = e;
}

```

Problem 4

```

typedef struct {
    enum { V_INT, V_DOUBLE, V_STRING } tag;
    union { int i; double d; char *s; } u;
} Value;

void print_value(Value v) {
    switch (v.tag) {
        case V_INT: printf("%d", v.u.i); break;
        case V_DOUBLE: printf("%g", v.u.d); break;
        case V_STRING: printf("%s", v.u.s); break;
    }
    putchar('\n');
}

```

Problem 5

This is the target problem. Use the `sym_insert` / `sym_lookup` / `sym_print` code from the 'Solve the target' section, changing `defn` to an `int` count. In the `insert` function, if the name already exists, increment count instead of replacing a definition string. The driver reads words in a loop with `scanf("%s", word)` and calls `insert` for each.